

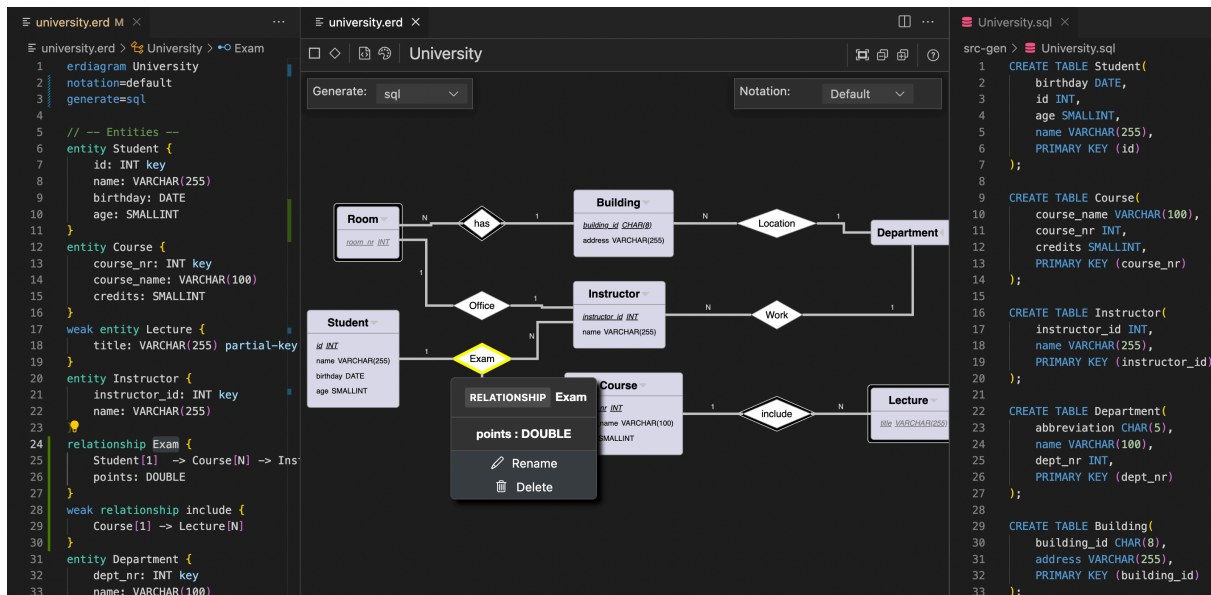
T3: SQL Import: SQL DDL → ER

Table of Contents

- [1. Context](#)
 - [2. Motivation](#)
 - [3. Goals](#)
 - [4. Technologies](#)
 - [5. Deliverables](#)
-

1. Context

The projects are situated in the context of **bigER**, an open-source **Entity-Relationship (ER) modeling tool** for **Visual Studio Code (VS Code)**. The central feature of bigER is its **hybrid modeling environment**, in which users can work with a **textual ER language** and a **graphical diagram view** in parallel while both representations remain fully synchronized (i.e., changes in one editor are immediately reflected in the other editor). To realize this behavior, the textual representation is the leading model and graphical edits are first translated into textual changes, which are then propagated back to the diagram. This hybrid design combines the strengths of both forms of modeling, as users can choose the textual editor for fast and efficient textual modeling, while the diagram view offers better usability and a more intuitive overview of the underlying model.

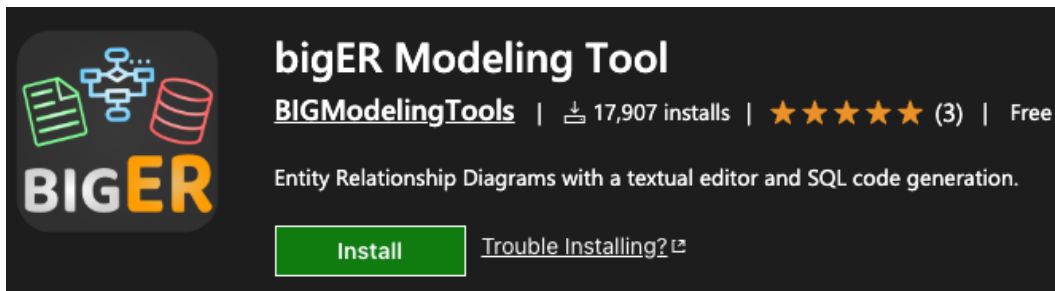


Screenshot of the bigER Modeling Tool

Technically, bigER is realized as a **client-server application**. The original implementation was developed with an **Xtext-based language server** (written in Xtend/Java), combined with **Sprotty** for diagram support and integrated into VS Code as an extension. In this architecture, Xtext provides the grammar-based language infrastructure and default language-server functionality (similar to langium), while Sprotty handles the generation and rendering of graphical models, interactive diagram behavior, and the connection to the VS Code client.

The original bigER implementation already went beyond a minimal editor. It provides a custom ER language with typical language-server features such as validation and rich editor support (via the Language Server Protocol (LSP)), a synchronized diagram view, and code generation for database-related artifacts. Over time, various theses and student projects extended bigER into a broader tool with additional capabilities such as **multiple notation support** and generation support for different target technologies (e.g., different SQL dialects, NoSQL).

Today, the tool has been downloaded more than 17,500 times, which indicates practical interest and relevance beyond an academic prototype. At the same time, an important limitation of the original implementation is its dependency on **Java** due to the Xtext-based server side. In practice, this can complicate installation and maintenance on different machines and operating systems (e.g., different Java versions or additional setup requirements).



bigER in the VS Code Marketplace

The current effort therefore focuses on a **reimplementation of bigER with Langium**. Langium is an open-source language engineering framework with first-class support for the **Language Server Protocol**, written in **TypeScript** and running on **Node.js**. Moving from Xtext to Langium is therefore not just a framework substitution as it means shifting the language-server side from a Java-based technology stack to a fully TypeScript-based one. This removes the Java dependency and aligns the implementation more closely with the surrounding web-based VS Code extension ecosystem.

At the same time, this migration is not a greenfield project (i.e., not starting from scratch). A minimal Langium-based prototype already exists and demonstrates the fundamental feasibility of the new approach, including the hybrid modeling view and selected basic functionality. The task in the individual projects is therefore not to implement a new ER tool from scratch, but to **systematically rebuild and extend important capabilities of bigER within the new architecture**. The original tool serves as a functional reference point, but its solutions should not simply be copied. Instead, the projects are expected to reassess how specific capabilities can be realized in a Langium-based setting, potentially in a cleaner, more robust, more configurable, and better testable way.

This is also what makes the topics academically interesting as they combine language engineering, model-driven development, tool architecture, and practical software engineering in the context of a real hybrid modeling tool. Each topic thus contributes to the broader goal of establishing a Langium-based successor to bigER that remains useful as a practical modeling tool inside VS Code while being easier to evolve, test, and maintain than the original Java-based implementation.

2. Motivation

Many users do not start from a conceptual ER model, but from an already existing database schema. In such cases, it is useful to **reverse-engineer SQL DDL into BIGER ER models** in order to document existing systems, analyze database structures, and support further evolution or forward engineering. This makes SQL import a natural counterpart to code generation.

At the same time, reverse engineering is inherently challenging. SQL schemas often contain dialect-specific constructs, implementation-oriented design decisions, and incomplete semantic information. Important conceptual aspects of an ER model, e.g., higher-level relationships or inheritance hierarchies, are not always represented explicitly in DDL and often have to be inferred heuristically. A useful importer must therefore be **robust to dialect variants, configurable in ambiguous cases, and explicit about uncertainty**.

The topic is motivated by the need to make the Langium-based bigER tool useful understanding existing databases and for forward engineering. It is technically interesting because it combines parsing, schema analysis, heuristic reconstruction, and model transformation.

3. Goals

The goal of this topic is to design and implement an **SQL import module** for the Langium-based bigER tool that transforms SQL DDL into ER models.

More specifically, the project should address the following goals:

1. SQL DDL → BIGER ER Mapping

- Parse DDL and reconstruct core ER concepts such as entities (tables), attributes (columns), keys (PK/UK), and relationships (FK). Extract cardinalities and optionality where they can be inferred, for example from nullability and uniqueness constraints.
- Preferably use a mature SQL parser library with dialect support. Alternatively, implement a clearly scoped DDL parser from scratch with limited syntax coverage.
- Apply heuristics for higher-level concepts such as:
 - detecting join tables as N:M relations,
 - distinguishing associative entities,
 - identifying possible inheritance patterns.

2. Dialect support and configuration

- Support at least **PostgreSQL** and **MySQL**. Handle relevant dialect differences such as identifier quoting, schema-qualified names, type aliases, and auto-increment variants.
- Provide configuration options for ambiguous cases, for example:
 - whether join tables should be collapsed,
 - whether names should be preserved or normalized,
 - how SQL types should be mapped to BIGER types.
- Clearly document which semantics cannot be reconstructed reliably and how uncertainty is represented.

3. Testing

- Implement **unit tests** for mapping logic, heuristic decisions, and dialect-specific handling.
- Include **golden file tests** for representative input-output pairs.
- Include **negative tests** for invalid configurations, unsupported constructs, or ambiguous cases.

4. Technologies

The following technologies are recommended as a starting point. They are **not mandatory choices**.

- **TypeScript / Node.js**: Core implementation environment of the Langium-based bigER tool.
- **Langium**: Relevant for creating and serializing BIGER-compatible ER models on the target side.
- **Existing `import` package code skeleton**: Intended as the main starting point for the implementation and should be extended accordingly.
- **SQL parser library with dialect support**: Preferably a mature parser that supports at least PostgreSQL and MySQL. If this is not feasible, a limited custom parser with clearly defined scope may be implemented.
- **Vitest, Jest, or similar**: Recommended for unit tests.

- **SQL + dialect knowledge:** Especially PostgreSQL and MySQL, including DDL syntax, constraints, types, quoting rules, and naming conventions.

5. Deliverables

The expected deliverables for this topic are:

- **Code contribution** in the group's designated repository / branch. This includes:
 - A working **SQL import implementation** that transforms SQL DDL into BIGER ER models. Support for at least **PostgreSQL** and **MySQL** input.
 - Integration of the importer into the repository, for example through a **CLI** and/or a **VS Code command** (to be decided).
 - A suitable **test suite**, including representative unit tests and output-based tests.
- **Documentation** of:
 - the implemented SQL-to-ER mapping rules,
 - supported dialect features,
 - configuration options,
 - and known limitations.
- A **final report** describing the implemented solution, design decisions, limitations, and recommendations for future work.